

🌾 Week 2 : Data Persistence with Core Data

Topic: Saving Data Permanently with Core Data

****Learning Objectives****

By the end of this week, students will be able to:

- Convert models to Core Data entities
- Set up Core Data stack (NSPersistentContainer)
- Understand @FetchRequest for automatic UI updates
- Perform basic CRUD operations (Create, Read, Update, Delete)
- Work with Managed Object Context

⚠️ Important Note: SwiftData vs Core Data

****SwiftData**** (iOS 17+ / Xcode 15+):

- Newer, simpler syntax
- Uses `@Model` macro
- Requires iOS 17 minimum

****Core Data**** (iOS 3+ / All Xcode versions):

- Mature, stable framework
- Uses `NSManagedObject` subclasses
- Works with iOS 13+

Since we target ****iOS 13+****, we'll use ****Core Data****.

⚠️ iOS 13+ API Rules (Important!)

Feature	❌ iOS 17+ Only	✅ iOS 13+ Correct
---	---	---
ViewModel	`@Observable class VM`	`class VM: ObservableObject`
State ViewModel	`@State var vm: VM`	`@StateObject var vm: VM`
Read ViewModel	`@Environment(VM.self)`	`@EnvironmentObject var vm: VM`
Pass ViewModel	`environment(vm)`	`environmentObject(vm)`
Navigation	`NavigationStack`	`NavigationView`

📖 Lesson 2.1: Setting Up Core Data in Xcode (45 minutes)

****2.1.1 Creating the Data Model File****

...

// Step 1: In Xcode, File → New → File...

```
// Step 2: Choose "Data Model" from Core Data section
// Step 3: Name it "SmartFarmerAssistantFinish.xcdatamodeld"
// Step 4: Save in your project folder
...
```

>  The model name in code must exactly match your .xcdatamodeld file name.

****2.1.2 Setting Up Core Data Stack****

>  ****Only ONE `NSPersistentContainer` should exist in the entire app.****
> If you create it in both `App` and `CoreDataManager`, Core Data loads
> the model twice → crash: **"Multiple NSEntityDescriptions claim the subclass"**.

****Correct pattern – use `CoreDataManager.shared` everywhere:****

```
```swift
// App/SmartFarmerAssistantFinishApp.swift
import SwiftUI
import CoreData

@main
struct SmartFarmerAssistantFinishApp: App {

 //  Reuse the single shared container – do NOT create a new one here
 let context = CoreDataManager.shared.context

 var body: some Scene {
 WindowGroup {
 MainTabView()
 .environment(\.managedObjectContext, context)
 }
 }
}
...`
```

### **\*\*2.1.3 Creating Core Data Manager (Helper Class)\*\***

```
```swift
// Utilities/CoreDataManager.swift
import Foundation
import CoreData

class CoreDataManager {
    static let shared = CoreDataManager()

    lazy var persistentContainer: NSPersistentContainer = {
        // Name must match your .xcdatamodeld file
        let container = NSPersistentContainer(name:
            "SmartFarmerAssistantFinish")
        container.loadPersistentStores { description, error in
            if let error = error {
                fatalError("Unable to load Core Data stack: \(error)")
            }
        }
    }()
}
```

```

        }
    }
    return container
}()

var context: NSManagedObjectContext {
    return persistentContainer.viewContext
}

func saveContext() {
    if context.hasChanges {
        do {
            try context.save()
        } catch {
            print("Error saving context: \(error)")
        }
    }
}
}
}

```

📖 Lesson 2.2: Creating Core Data Entities (45 minutes)

****2.2.1 Creating Entities in Data Model Editor****

Open `SmartFarmerAssistantFinish.xcdatamodeld` and create these entities:

****Entity: Transaction****

...

Attributes:

- amount: Double
- date: Date
- note: String
- type: String
- category: String
- id: UUID (default value: \$(UUID))

...

****Entity: FarmActivity****

...

Attributes:

- title: String
- activityType: String
- date: Date
- notes: String
- isCompleted: Boolean
- reminderEnabled: Boolean
- id: UUID

...

****Entity: Pest****

...

Attributes:

- name: String
- pestType: String
- symptoms: String
- treatment: String
- prevention: String
- imageName: String
- isFavorite: Boolean
- id: UUID

...

****Entity: JournalEntry****

...

Attributes:

- date: Date
- content: String
- weather: String
- photoData: Binary Data (allows external storage)
- location: String
- id: UUID

...

****2.2.2 Generating NSManagedObject Subclasses****

...

```
// Step 1: Click on the entity in data model editor
// Step 2: In the right panel (Data Model Inspector):
//          Set "Codegen" to "Manual/None"
// Step 3: Editor → Create NSManagedObject Subclass...
// Step 4: Xcode will generate files like:
//          - Transaction+CoreDataClass.swift
//          - Transaction+CoreDataProperties.swift
```

...

>  If you write the files manually, set Codegen to "Manual/None" to avoid
> the "Multiple commands produce" build error.

****2.2.3 Manual Model Classes****

```
```swift
// Models/Transaction+CoreDataClass.swift
import Foundation
import CoreData

@objc(Transaction)
public class Transaction: NSManagedObject {

}

// Models/Transaction+CoreDataProperties.swift
import Foundation
import CoreData

extension Transaction {
```

```

@nonobjc public class func fetchRequest() ->
 NSFetchRequest<Transaction> {
 return NSFetchRequest<Transaction>(entityName: "Transaction")
 }

@NSManaged public var amount: Double
@NSManaged public var date: Date?
@NSManaged public var note: String?
@NSManaged public var type: String?
@NSManaged public var category: String?
@NSManaged public var id: UUID?
}

extension Transaction: Identifiable {}
```



```

```swift
// Models/TransactionType.swift
enum ExpenseCategory: String, CaseIterable {
    case seeds = "គ្រាប់ពូជ"
    case fertilizer = "ដី"
    case labor = "កម្លាំងពលកម្ម"
    case tools = "ឧបករណ៍"
    case other = "ផ្សេងៗ"
}

enum IncomeCategory: String, CaseIterable {
    case vegetable = "បន្លែ"
    case fruit = "ផ្លែឈើ"
    case grain = "ស្រូវ-ដំណាំ"
    case livestock = "សត្វ"
    case other = "ផ្សេងៗ"
}
```

```


```

Lesson 2.3: Understanding @FetchRequest for UI Updates (45 minutes)

****2.3.1 What is @FetchRequest?****

In Core Data, `@FetchRequest` automatically watches for changes in your data and updates the UI when changes occur.

```

```swift
import SwiftUI
import CoreData

```

```

struct TransactionListView: View {
 // Automatically fetches and watches for changes
 @FetchRequest(
 entity: Transaction.entity(),
 sortDescriptors: [NSSortDescriptor(keyPath: \Transaction.date,
 ascending: false)]
) var transactions: FetchedResults<Transaction>

 // Filtered fetch request
 @FetchRequest(
 entity: Transaction.entity(),
 sortDescriptors: [],
 predicate: NSPredicate(format: "type == %@", "expense")
) var expenses: FetchedResults<Transaction>

 var body: some View {
 List {
 ForEach(transactions, id: \.self) { transaction in
 VStack(alignment: .leading) {
 Text(transaction.note ?? "No note")
 .font(.headline)
 Text("Amount: \(transaction.amount)")
 .font(.subheadline)
 }
 }
 }
 }
}

```

### **\*\*2.3.2 Dynamic Fetch Requests with NSPredicate\*\***

#### **\*\*How it works – process flow:\*\***

```

...
User taps Picker → filterType changes → FilteredTransactionList.init()
runs again
 |
 ▼
New NSPredicate built → FetchRequest rebuilt → SQLite query runs
 |
 ▼
Only matching rows returned → List refreshes automatically
...

```

#### **\*\*Why NSPredicate is better than Swift `.filter {}`:\*\***

Swift `.filter { }`	NSPredicate
Where filtering happens	In memory (after all rows loaded)   In the database (SQLite)
Performance	Loads ALL rows first   Only loads matching rows
Use case	Simple / small data   Recommended for Core Data

**\*\*When to use `DynamicFilterView` vs delete it:\*\***

Situation	Decision
You want a reusable filter picker + list as one component	Keep `DynamicFilterView`
Your parent view (e.g. `FinanceTabView`) already has its own Picker + summary cards	<b>**Delete `DynamicFilterView`**</b> – use `FilteredTransactionList` directly

> In this project, `FinanceTabView` owns the Picker and summary cards, so  
> `DynamicFilterView` is redundant and has been removed.  
> Only `FilteredTransactionList` is kept – it is the reusable part.

**\*\*Complete implementation – `Views/DynamicFilterView.swift` (contains only `FilteredTransactionList`):\*\***

```
```swift
import SwiftUI
import CoreData

// FilteredTransactionList – reusable component used by FinanceTabView
// Filters at database level using NSPredicate
// Handles its own delete; calls onTap so parent can open edit sheet
struct FilteredTransactionList: View {
    @Environment(\.managedObjectContext) private var viewContext

    let viewModel: FarmViewModel
    let onTap: (Transaction) -> Void // parent sets selectedTransaction

    var fetchRequest: FetchRequest<Transaction>
    var transactions: FetchedResults<Transaction> {
        fetchRequest.wrappedValue
    }

    init(filterType: String,
         viewModel: FarmViewModel,
         onTap: @escaping (Transaction) -> Void) {
        self.viewModel = viewModel
        self.onTap = onTap

        // "all" → nil predicate (no WHERE clause → fetch everything)
        // "expense" → NSPredicate(format: "type == 'expense'")
        // "income" → NSPredicate(format: "type == 'income'")
        let predicate: NSPredicate? = filterType == "all"
            ? nil
            : NSPredicate(format: "type == %@", filterType)

        self.fetchRequest = FetchRequest(
            entity: Transaction.entity(),
            sortDescriptors: [NSSortDescriptor(keyPath: \Transaction.date,
                                                ascending: false)],
            predicate: predicate // ← passed directly to SQLite
        )
    }
}
```

```

var body: some View {
    List {
        ForEach(transactions, id: \.self) { transaction in
            TransactionRowView(transaction: transaction, viewModel:
                viewModel)
                .onTapGesture { onTap(transaction) } // notify parent
                    to open edit
        }
        .onDelete(perform: deleteTransactions) // swipe-to-delete
    }
}

// Delete is handled here because only this view has access to
transactions[]
private func deleteTransactions(offsets: IndexSet) {
    for index in offsets {
        viewContext.delete(transactions[index])
    }
    try? viewContext.save()
}
}
...

```

****How to use `FilteredTransactionList` in `FinanceTabView`:****

```

```swift
// In FinanceTabView body – replaces the manual ForEach + .filter {}
@State private var selectedTransaction: Transaction? = nil

// Inside VStack, after the Picker:
FilteredTransactionList(
 filterType: filterType, // drives the NSPredicate
 viewModel: viewModel,
 onTap: { selectedTransaction = $0 } // tap row → open edit sheet
)
.environment(\.managedObjectContext, viewContext)
.sheet(item: $selectedTransaction) { transaction in
 EditTransactionView(transaction: transaction)
 .environment(\.managedObjectContext, viewContext)
 .environmentObject(viewModel)
}
...

```

> **\*\*Note:\*\*** `FinanceTabView` keeps its own `@FetchRequest` for **\*\*all\*\*** transactions  
 > (no predicate) only to calculate the summary card totals (income / expense / balance).  
 > `FilteredTransactionList` uses a separate `@FetchRequest` for the filtered list.

### **\*\*2.3.3 ViewModel with ObservableObject (iOS 13+)\*\***

```

```swift

```



```

// ✅ Use ObservableObject for iOS 13+ compatibility
// ❌ Do NOT use @Observable (requires iOS 17)
class FarmViewModel: ObservableObject {
    private var context: NSManagedObjectContext

    init(context: NSManagedObjectContext = CoreDataManager.shared.context) {
        self.context = context
    }

    // MARK: – CRUD Operations

    func addTransaction(amount: Double, note: String, type: String,
        category: String) {
        let transaction = Transaction(context: context)
        transaction.amount = amount
        transaction.date = Date()
        transaction.note = note
        transaction.type = type
        transaction.category = category
        transaction.id = UUID()
        saveContext()
    }

    func updateTransaction(_ transaction: Transaction,
        amount: Double? = nil,
        note: String? = nil) {
        if let amount = amount { transaction.amount = amount }
        if let note = note { transaction.note = note }
        saveContext()
    }

    func deleteTransaction(_ transaction: Transaction) {
        context.delete(transaction)
        saveContext()
    }

    func deleteAllTransactions() {
        let fetchRequest: NSFetchedRequest<NSFetchRequestResult> =
            Transaction.fetchRequest()
        let batchDeleteRequest = NSBatchDeleteRequest(fetchRequest:
            fetchRequest)
        do {
            try context.execute(batchDeleteRequest)
            saveContext()
        } catch {
            print("Error deleting all transactions: \(error)")
        }
    }

    private func saveContext() {
        CoreDataManager.shared.saveContext()
    }

    // MARK: – Helper Methods

```

```

func formatCurrency(_ amount: Double) -> String {
    let formatter = NumberFormatter()
    formatter.numberStyle = .currency
    formatter.locale = Locale(identifier: "en_US")
    formatter.maximumFractionDigits = 2
    return formatter.string(from: NSNumber(value: amount)) ?? "$0.00"
}

func calculateTotalBalance(transactions: FetchedResults<Transaction>)
-> Double {
    var total: Double = 0
    for transaction in transactions {
        if transaction.type == "income" {
            total += transaction.amount
        } else {
            total -= transaction.amount
        }
    }
    return total
}
}
...

```

Lesson 2.4: Building the Finance Tab with Core Data (45 minutes)

****2.4.1 Transaction Row View****

```

```swift
// Views/TransactionRowView.swift
import SwiftUI

struct TransactionRowView: View {
 let transaction: Transaction
 let viewModel: FarmViewModel

 var body: some View {
 HStack {
 Circle()
 .fill(transaction.type == "expense" ? Color.red :
 Color.green)
 .frame(width: 40, height: 40)
 .overlay(
 Image(systemName: transaction.type == "expense" ?
 "arrow.down" : "arrow.up")
 .foregroundColor(.white)
)

 VStack(alignment: .leading, spacing: 4) {
 Text(transaction.category ?? "")
 .font(.headline)
 }
 }
 }
}

```

```

 if let note = transaction.note, !note.isEmpty {
 Text(note)
 .font(.caption)
 .foregroundColor(.gray)
 }

 if let date = transaction.date {
 Text(date, style: .date)
 .font(.caption2)
 .foregroundColor(.gray)
 }
 }

 Spacer()

 Text(viewModel.formatCurrency(transaction.amount))
 .font(.headline)
 .foregroundColor(transaction.type == "expense" ? .red :
 .green)
 }
 .padding(.vertical, 4)
}
}
...

```

#### **\*\*2.4.2 Add Transaction Sheet\*\***

```

```swift
// Views/AddTransactionView.swift
import SwiftUI

struct AddTransactionView: View {
    @Environment(\.presentationMode) var presentationMode
    // ✅ @EnvironmentObject for iOS 13+ (❌ not
    @Environment(FarmViewModel.self))
    @EnvironmentObject private var viewModel: FarmViewModel
    @Environment(\.managedObjectContext) private var viewContext

    @State private var amount = ""
    @State private var note = ""
    @State private var selectedType = "expense"
    @State private var selectedExpenseCategory =
        ExpenseCategory.other.rawValue
    @State private var selectedIncomeCategory =
        IncomeCategory.other.rawValue

    let types = ["expense", "income"]

    var body: some View {
        // ✅ NavigationView for iOS 13+ (❌ not UINavigationController)
        NavigationView {
            Form {

```

```

Section(header: Text("ចំនួនទឹកប្រាក់")) {
    TextField("0", text: $amount)
        .keyboardType(.numberPad)
}

Section(header: Text("ប្រភេទ")) {
    Picker("ប្រភេទ", selection: $selectedType) {
        ForEach(types, id: \.self) { type in
            Text(type == "expense" ? "ចំណាយ" :
                "ចំណូល").tag(type)
        }
    }
    .pickerStyle(SegmentedPickerStyle())
}

Section(header: Text("ប្រភេទធន")) {
    if selectedType == "expense" {
        Picker("ជ្រើសរើស", selection:
            $selectedExpenseCategory) {
            ForEach(ExpenseCategory.allCases, id:
                \.rawValue) { category in
                Text(category.rawValue).tag(category
                    .rawValue)
            }
        }
    } else {
        Picker("ជ្រើសរើស", selection:
            $selectedIncomeCategory) {
            ForEach(IncomeCategory.allCases, id:
                \.rawValue) { category in
                Text(category.rawValue).tag(category
                    .rawValue)
            }
        }
    }
}

Section(header: Text("កំណត់ចំណាំ")) {
    TextField("បញ្ចូលកំណត់ចំណាំ...", text: $note)
}

.navigationTitle("បន្ថែមប្រតិបត្តិការ")
.navigationBarItems(
    leading: Button("បោះបង់") {
        presentationMode.wrappedValue.dismiss()
    },
    trailing: Button("រក្សាទុក") {

```

```

        saveTransaction()
    }
    .disabled(amount.isEmpty)
)
}
}

private func saveTransaction() {
    guard let amountValue = Double(amount) else { return }
    let category = selectedType == "expense" ? selectedExpenseCategory
        : selectedIncomeCategory

    let transaction = Transaction(context: viewContext)
    transaction.amount = amountValue
    transaction.date = Date()
    transaction.note = note
    transaction.type = selectedType
    transaction.category = category
    transaction.id = UUID()

    do {
        try viewContext.save()
        presentationMode.wrappedValue.dismiss()
    } catch {
        print("Error saving transaction: \(error)")
    }
}
}
...

```

****2.4.3 Complete Finance Tab View****

```

```swift
// Views/FinanceTabView.swift
import SwiftUI
import CoreData

struct FinanceTabView: View {
 //  @EnvironmentObject for iOS 13+ ( not
 @Environment(FarmViewModel.self))
 @EnvironmentObject private var viewModel: FarmViewModel
 @Environment(\.managedObjectContext) private var viewContext

 @State private var showingAddTransaction = false
 @State private var filterType = "all"

 @FetchRequest(
 entity: Transaction.entity(),
 sortDescriptors: [NSSortDescriptor(keyPath: \Transaction.date,
 ascending: false)]
) var allTransactions: FetchedResults<Transaction>

 var displayedTransactions: [Transaction] {
 switch filterType {

```

```

 case "expense": return allTransactions.filter { $0.type ==
 "expense" }
 case "income": return allTransactions.filter { $0.type == "income"
 }
 default: return Array(allTransactions)
 }
}

var totalIncome: Double {
 allTransactions.filter { $0.type == "income" }.reduce(0) { $0 +
 $1.amount }
}

var totalExpense: Double {
 allTransactions.filter { $0.type == "expense" }.reduce(0) { $0 +
 $1.amount }
}

var balance: Double { totalIncome - totalExpense }

var body: some View {
 // ✅ NavigationView for iOS 13+ (❌ not UINavigationController)
 NavigationView {
 VStack(spacing: 0) {
 HStack(spacing: 12) {
 SummaryCard(title: "ចំណូល", amount: totalIncome, color:
 .green, icon: "arrow.up.circle.fill")
 SummaryCard(title: "ចំណាយ", amount: totalExpense,
 color: .red, icon: "arrow.down.circle.fill")
 SummaryCard(title: "សមតុល្យ", amount: balance, color:
 .blue, icon: "equal.circle.fill")
 }
 .padding()

 Picker("ត្រូវបាន", selection: $filterType) {
 Text("ទាំងអស់").tag("all")
 Text("ចំណូល").tag("income")
 Text("ចំណាយ").tag("expense")
 }
 .pickerStyle(SegmentedPickerStyle())
 .padding(.horizontal)

 List {
 ForEach(displayedTransactions, id: \.self) {
 transaction in
 TransactionRowView(transaction: transaction,
 viewModel: viewModel)
 }
 .onDelete(perform: deleteTransactions)
 }
 }
 }
}

```

```

 }
 .navigationTitle("កំណត់ត្រាចំណាយចំណូល")
 .toolbar {
 ToolbarItem(placement: .navigationBarTrailing) {
 Button(action: { showingAddTransaction = true }) {
 Image(systemName: "plus")
 }
 }
 }
 .sheet(isPresented: $showingAddTransaction) {
 AddTransactionView()
 .environment(\.managedObjectContext, viewModel.environmentObject(viewModel))
 }
}

private func deleteTransactions(offsets: IndexSet) {
 for index in offsets {
 viewModel.delete(displayedTransactions[index])
 }
 try? viewModel.save()
}

struct SummaryCard: View {
 let title: String
 let amount: Double
 let color: Color
 let icon: String
 // ✅ @EnvironmentObject for iOS 13+ (❌ not
 @Environment(FarmViewModel.self))
 @EnvironmentObject private var viewModel: FarmViewModel

 var body: some View {
 VStack(alignment: .leading, spacing: 8) {
 HStack {
 Image(systemName: icon)
 .foregroundColor(color)
 Text(title)
 .font(.caption)
 .foregroundColor(.gray)
 }
 Text(viewModel.formatCurrency(amount))
 .font(.headline)
 .foregroundColor(color)
 }
 .frame(maxWidth: .infinity, alignment: .leading)
 .padding()
 .background(color.opacity(0.1))
 .cornerRadius(10)
 }
}

```

---

## ## 📖 Lesson 2.5: Update (Edit) Transaction + Currency Formatting (45 minutes)

---

### ### 2.5.1 Understanding the Update Flow

Before writing code, understand the full process:

```

USER TAPS A ROW



FinanceTabView sets selectedTransaction = transaction



.sheet(item: \$selectedTransaction) triggers



EditTransactionView opens (pre-filled with existing data)



User edits fields and taps "စုစုပေါင်း" (Save)



viewModel.updateTransaction(...) called



CoreDataManager.shared.saveContext() persists to disk



@FetchRequest in FinanceTabView auto-refreshes the list

```

---

### ### 2.5.2 Step 1 – Update `updateTransaction` in ViewModel

The ViewModel is the single place that performs all Core Data writes. Accept all editable fields so the UI does not touch Core Data directly.

```swift

// ViewModels/FarmViewModel.swift

```
// Update – overwrites all editable fields at once
func updateTransaction(_ transaction: Transaction,
                      amount: Double,
                      note: String,
                      type: String,
                      category: String) {
```



```

        transaction.amount    = amount
        transaction.note      = note
        transaction.type       = type
        transaction.category   = category
        saveContext()          // ← persists change to disk
    }

```

```

private func saveContext() {
    CoreDataManager.shared.saveContext()
}
...

```

> ****Why call `saveContext()` at the end?****

> Changing a property on an `NSManagedObject` only marks it as "dirty" in memory.
 > `saveContext()` is what actually writes the change to the SQLite file on disk.
 > Without it, the change is lost when the app restarts.

2.5.3 Step 2 – Update `formatCurrency` to USD

```

```swift
// ViewModels/FarmViewModel.swift

func formatCurrency(_ amount: Double) -> String {
 let formatter = NumberFormatter()
 formatter.numberStyle = .currency
 formatter.locale = Locale(identifier: "en_US") // USD $
 formatter.maximumFractionDigits = 2
 return formatter.string(from: NSNumber(value: amount)) ?? "$0.00"
}
...

```

Locale identifier	Output example
"en_US"	\$1,234.56
"km_KH"	1.234,56 \$
"ja_JP"	¥1,235

> Change `Locale(identifier:)` to switch currency at any time.

---

### ### 2.5.4 Step 3 – Add Edit State to `FinanceTabView`

Add one `@State` property to track which transaction was tapped, then attach a sheet that opens when it is non-nil.

```

```swift
// Views/FinanceTabView.swift (changes only)

@State private var showingAddTransaction = false

```

```

@State private var selectedTransaction: Transaction? = nil    // ← NEW

// Inside the List:
ForEach(displayedTransactions, id: \.self) { transaction in
    TransactionRowView(transaction: transaction, viewModel: viewModel)
        .onTapGesture { selectedTransaction = transaction } // ← tap to
        edit
}
.onDelete(perform: deleteTransactions)

// Attach edit sheet to the List (after .onDelete):
.sheet(item: $selectedTransaction) { transaction in
    EditTransactionView(transaction: transaction)
        .environment(\.managedObjectContext, viewContext)
        .environmentObject(viewModel)
}
...

```

```

> Why `.sheet(item:)` instead of `.sheet(isPresented:)?`
>
> | | `.sheet(isPresented: $bool)` | `.sheet(item: $optional)` |
> |---|---|---|
> | Trigger | Bool toggle | Any `Identifiable` going non-nil |
> | Passes data | Must use a separate `@State` | Item passed directly into
  closure |
> | Best for | Add forms | Edit forms (need the item) |

```

2.5.5 Step 4 – Create `EditTransactionView`

Pre-fill every field from the existing transaction using
`State(initialValue:)`.

```

```swift
// Views/EditTransactionView.swift
import SwiftUI

struct EditTransactionView: View {
 @Environment(\.presentationMode) var presentationMode
 @EnvironmentObject private var viewModel: FarmViewModel

 let transaction: Transaction // the record being edited

 // Pre-fill state from the existing transaction
 @State private var amount: String
 @State private var note: String
 @State private var selectedType: String
 @State private var selectedExpenseCategory: String
 @State private var selectedIncomeCategory: String

 let types = ["expense", "income"]

 init(transaction: Transaction) {
 self.transaction = transaction
 }
}

```

```

 _amount = State(initialValue: String(transaction.amount))
 _note = State(initialValue: transaction.note ?? "")
 _selectedType = State(initialValue: transaction.type ?? "expense")
 _selectedExpenseCategory = State(initialValue:
 transaction.type == "expense"
 ? (transaction.category ?? ExpenseCategory.other.rawValue)
 : ExpenseCategory.other.rawValue)
 _selectedIncomeCategory = State(initialValue:
 transaction.type == "income"
 ? (transaction.category ?? IncomeCategory.other.rawValue)
 : IncomeCategory.other.rawValue)
 }

 var body: some View {
 NavigationView {
 Form {
 Section(header: Text("ចំនួនទឹកប្រាក់")) {
 TextField("0.00", text: $amount)
 .keyboardType(.decimalPad)
 }

 Section(header: Text("ប្រភេទ")) {
 Picker("ប្រភេទ", selection: $selectedType) {
 ForEach(types, id: \.self) { type in
 Text(type == "expense" ? "ចំណាយ" :
 "ចំណូល").tag(type)
 }
 }
 .pickerStyle(SegmentedPickerStyle())
 }

 Section(header: Text("ប្រភេទរង")) {
 if selectedType == "expense" {
 Picker("ជ្រើសរើស", selection:
 $selectedExpenseCategory) {
 ForEach(ExpenseCategory.allCases, id:
 \.rawValue) { cat in
 Text(cat.rawValue).tag(cat.rawValue)
 }
 }
 } else {
 Picker("ជ្រើសរើស", selection:
 $selectedIncomeCategory) {
 ForEach(IncomeCategory.allCases, id:
 \.rawValue) { cat in
 Text(cat.rawValue).tag(cat.rawValue)
 }
 }
 }
 }
 }
 }
 }

```

```

 Section(header: Text("កំណត់ចំណាំ")) {
 TextField("បញ្ចូលកំណត់ចំណាំ...", text: $note)
 }
 }
 .navigationTitle("កែប្រែប្រតិបត្តិការ")
 .navigationBarItems(
 leading: Button("បោះបង់") {
 presentationMode.wrappedValue.dismiss()
 },
 trailing: Button("រក្សាទុក") {
 saveChanges()
 }
 .disabled(amount.isEmpty)
)
}

private func saveChanges() {
 guard let amountValue = Double(amount) else { return }
 let category = selectedType == "expense"
 ? selectedExpenseCategory
 : selectedIncomeCategory

 // Call ViewModel – never write to Core Data directly from a View
 viewModel.updateTransaction(
 transaction,
 amount: amountValue,
 note: note,
 type: selectedType,
 category: category
)
 presentationMode.wrappedValue.dismiss()
}
}
...

```

```

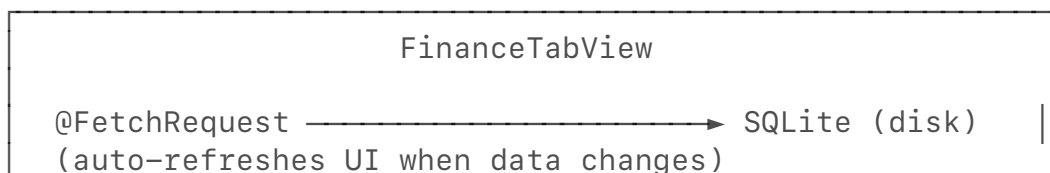
> **Key pattern – `State(initialValue:)` in `init`:**
> Normal `@State private var x = "value"` cannot read from a property.
> Use `_x = State(initialValue: someProperty)` inside `init` to pre-fill
> `@State` from an external value.

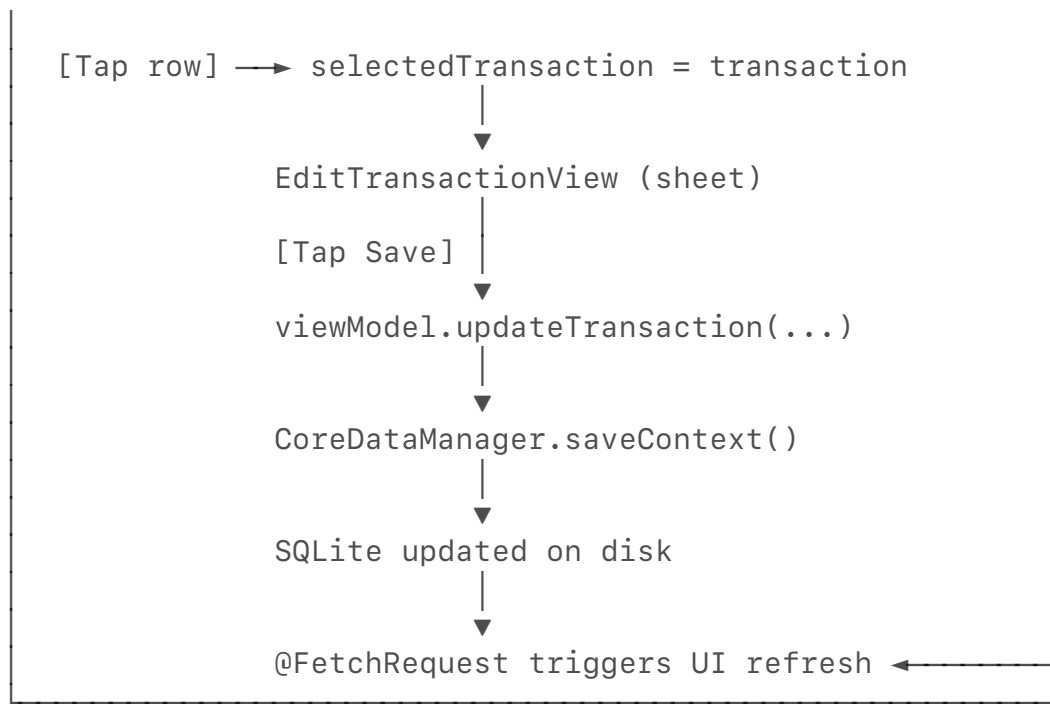
```

---

### ### 2.5.6 Full Data Flow Diagram

...





...

---

### ### 2.5.7 Common Update Mistakes

Mistake	Problem	Fix
---	---	---
Edit <code>transaction.x</code> directly in View	Bypasses ViewModel	Always call <code>viewModel.updateTransaction(...)</code>
Forget <code>saveContext()</code>	Change lost on restart	Always end with <code>saveContext()</code>
Use <code>@State var x = ""</code> in edit form	Fields start empty	Use <code>State(initialValue:)</code> in <code>init</code>
Use <code>.sheet(isPresented:)</code> for edit	Can't pass the item	Use <code>.sheet(item: \$selectedTransaction)</code>

---

### ### 2.5.8 MainTabView – Complete Setup

```

```swift
// Views/MainTabView.swift
import SwiftUI
import CoreData

struct MainTabView: View {
    @StateObject private var viewModel: FarmViewModel
    @State private var selectedTab = 0
    @Environment(\.managedObjectContext) private var viewContext

    init() {
        _viewModel = StateObject(wrappedValue: FarmViewModel(context:
            CoreDataManager.shared.context))
    }

```

```

var body: some View {
    TabView(selection: $selectedTab) {
        FinanceTabView()
            .tabItem { Label("ហិរញ្ញវត្ថុ", systemImage: "dollarsign.circle")
            }
            .tag(0)
            .environment(\.managedObjectContext, viewModel)

        CalendarTabView()
            .tabItem { Label("ប្រតិទិន", systemImage: "calendar") }
            .tag(1)
            .environment(\.managedObjectContext, viewModel)

        PestGuideTabView()
            .tabItem { Label("សត្វល្អិត", systemImage: "ant") }
            .tag(2)
            .environment(\.managedObjectContext, viewModel)

        JournalTabView()
            .tabItem { Label("កំណត់ហេតុ", systemImage: "book") }
            .tag(3)
            .environment(\.managedObjectContext, viewModel)
    }
    .environmentObject(viewModel)
}
}
...

---

```

2.5.9 Debug Helper for Core Data

```

```swift
// Utilities/CoreDataDebug.swift
import CoreData
import Foundation

struct CoreDataDebug {
 static func printDatabaseLocation() {
 let urls = FileManager.default.urls(for: .documentDirectory, in:
 .userDomainMask)
 print("Document Directory: \(urls[0])")
 }

 static func countEntities(context: NSManagedObjectContext) {
 do {
 let count = try context.count(for: Transaction.fetchRequest())
 print("Transaction count: \(count)")
 } catch {
 print("Error counting entities: \(error)")
 }
 }
}

```

```
}
...
```

---

## ## 📖 Lesson 2.6: Calendar Tab – FarmActivity CRUD (45 minutes)

### \*\*2.6.1 CalendarTabView\*\*

```
```swift  
// Views/CalendarTabView.swift  
import SwiftUI  
import CoreData  
  
struct CalendarTabView: View {  
    @EnvironmentObject private var viewModel: FarmViewModel  
    @Environment(\.managedObjectContext) private var viewContext  
  
    @FetchRequest(  
        entity: FarmActivity.entity(),  
        sortDescriptors: [NSSortDescriptor(keyPath: \FarmActivity.date,  
            ascending: true)]  
    ) var activities: FetchedResults<FarmActivity>  
  
    @State private var showingAddActivity = false  
  
    var body: some View {  
        NavigationView {  
            List {  
                ForEach(activities, id: \.self) { activity in  
                    ActivityRowView(activity: activity, viewModel:  
                        viewModel)  
                }  
                .onDelete(perform: deleteActivities)  
            }  
            .navigationTitle("ប្រតិទិនកសិកម្ម")  
            .toolbar {  
                ToolbarItem(placement: .navigationBarTrailing) {  
                    Button(action: { showingAddActivity = true }) {  
                        Image(systemName: "plus")  
                    }  
                }  
            }  
            .sheet(isPresented: $showingAddActivity) {  
                AddActivityView()  
                    .environment(\.managedObjectContext, viewContext)  
                    .environmentObject(viewModel)  
            }  
        }  
    }  
  
    private func deleteActivities(offsets: IndexSet) {
```

```

        for index in offsets {
            viewContext.delete(activities[index])
        }
        try? viewContext.save()
    }
}
...

```

****2.6.2 ActivityRowView****

```

```swift
struct ActivityRowView: View {
 let activity: FarmActivity
 let viewModel: FarmViewModel

 var body: some View {
 HStack(spacing: 12) {
 Image(systemName: activity.isCompleted ?
 "checkmark.circle.fill" : "circle")
 .foregroundColor(activity.isCompleted ? .green : .gray)
 .font(.title2)
 .onTapGesture {
 activity.isCompleted.toggle()
 try? activity.managedObjectContext?.save()
 }

 VStack(alignment: .leading, spacing: 4) {
 Text(activity.title ?? "")
 .font(.headline)
 .strikethrough(activity.isCompleted)

 Text(activity.activityType ?? "")
 .font(.caption)
 .foregroundColor(.blue)

 if let date = activity.date {
 Text(viewModel.formatDate(date))
 .font(.caption2)
 .foregroundColor(.gray)
 }
 }

 Spacer()

 if activity.reminderEnabled {
 Image(systemName: "bell.fill")
 .foregroundColor(.orange)
 .font(.caption)
 }
 }
 .padding(.vertical, 4)
 }
}
...

```



### **\*\*2.6.3 AddActivityView\*\***

```
```swift
struct AddActivityView: View {
    @Environment(\.presentationMode) var presentationMode
    @Environment(\.managedObjectContext) private var viewContext

    @State private var title = ""
    @State private var activityType = "ដាំដំណាំ"
    @State private var notes = ""
    @State private var date = Date()
    @State private var reminderEnabled = false

    let activityTypes = ["ដាំដំណាំ", "ស្រោចទឹក", "បាញ់ថ្នាំ", "ប្រូតកាត់", "ផ្សេងៗ"]

    var body: some View {
        NavigationView {
            Form {
                Section(header: Text("ព័ត៌មានសកម្មភាព")) {
                    TextField("ចំណងជើង", text: $title)
                    Picker("ប្រភេទ", selection: $activityType) {
                        ForEach(activityTypes, id: \.self) {
                            Text($0).tag($0)
                        }
                    }
                    DatePicker("កាលបរិច្ឆេទ", selection: $date,
                        displayedComponents: .date)
                }
                Section(header: Text("កំណត់ចំណាំ")) {
                    TextField("បញ្ចូលកំណត់ចំណាំ...", text: $notes)
                }
                Section {
                    Toggle("បើកការរំលឹក", isOn: $reminderEnabled)
                }
            }
            .navigationTitle("បន្ថែមសកម្មភាព")
            .navigationBarItems(
                leading: Button("បោះបង់") {
                    presentationMode.wrappedValue.dismiss()
                },
                trailing: Button("រក្សាទុក") { saveActivity() }
                    .disabled(title.isEmpty)
            )
        }
    }

    private func saveActivity() {
        let activity = FarmActivity(context: viewContext)
        activity.id = UUID()
        activity.title = title
    }
}
```

```

        activity.activityType = activityType
        activity.notes = notes
        activity.date = date
        activity.isCompleted = false
        activity.reminderEnabled = reminderEnabled
        try? viewContext.save()
        presentationMode.wrappedValue.dismiss()
    }
}
...

```

📖 Lesson 2.7: Pest Guide Tab – Read + Favorite (45 minutes)

2.7.1 PestGuideTabView

```

```swift
// Views/PestGuideTabView.swift
import SwiftUI
import CoreData

struct PestGuideTabView: View {
 @EnvironmentObject private var viewModel: FarmViewModel
 @Environment(\.managedObjectContext) private var viewContext

 @FetchRequest(
 entity: Pest.entity(),
 sortDescriptors: [NSSortDescriptor(keyPath: \Pest.name, ascending:
 true)]
) var pests: FetchedResults<Pest>

 @State private var showingAddPest = false
 @State private var searchText = ""

 var displayedPests: [Pest] {
 if searchText.isEmpty { return Array(pests) }
 return pests.filter {
 ($0.name ?? "").localizedCaseInsensitiveContains(searchText) ||
 ($0.pestType ?? "").localizedCaseInsensitiveContains(searchText)
 }
 }

 var body: some View {
 NavigationView {
 List {
 ForEach(displayedPests, id: \.self) { pest in
 NavigationLink(destination: PestDetailView(pest: pest))
 {
 PestRowView(pest: pest)
 }
 }
 }
 .onDelete(perform: deletePests)
 }
 }
}

```

```

 }
 .navigationTitle("មគ្គុទ្ទេសសត្វល្អិត")
 .toolbar {
 ToolbarItem(placement: .navigationBarTrailing) {
 Button(action: { showingAddPest = true }) {
 Image(systemName: "plus")
 }
 }
 }
 .sheet(isPresented: $showingAddPest) {
 AddPestView()
 .environment(\.managedObjectContext, viewModel.environment)
 }
}

private func deletePests(offsets: IndexSet) {
 for index in offsets {
 viewModel.deletePest(at: index)
 }
 try? viewModel.save()
}
}
...

```

## **\*\*2.7.2 PestRowView + PestDetailView\*\***

```

```swift
struct PestRowView: View {
    let pest: Pest

    var body: some View {
        HStack(spacing: 12) {
            Image(systemName: "ant.fill")
                .foregroundColor(.red)
                .font(.title2)
                .frame(width: 40, height: 40)
                .background(Color.red.opacity(0.1))
                .cornerRadius(8)

            VStack(alignment: .leading, spacing: 4) {
                Text(pest.name ?? "").font(.headline)
                Text(pest.pestType ??
                    "").font(.caption).foregroundColor(.gray)
            }

            Spacer()

            if pest.isFavorite {
                Image(systemName: "star.fill").foregroundColor(.yellow)
            }
        }
        .padding(.vertical, 4)
    }
}

```



```

        Picker("ប្រភេទ", selection: $pestType) {
            ForEach(pestTypes, id: \.self) { Text($0).tag($0) }
        }
    Section(header: Text("រោគសញ្ញា")) { TextField("ពិពណ៌នា...",
        text: $symptoms) }
    Section(header: Text("វិធីព្យាបាល")) { TextField("ពិពណ៌នា...",
        text: $treatment) }
    Section(header: Text("វិធីការពារ")) { TextField("ពិពណ៌នា...",
        text: $prevention) }
}
.navigationTitle("បន្ថែមសត្វល្អិត")
.navigationBarItems(
    leading: Button("បោះបង់") {
        presentationMode.wrappedValue.dismiss() },
    trailing: Button("រក្សាទុក") { savePest()
        }.disabled(name.isEmpty)
)
}
}

private func savePest() {
    let pest = Pest(context: viewContext)
    pest.id = UUID()
    pest.name = name
    pest.pestType = pestType
    pest.symptoms = symptoms
    pest.treatment = treatment
    pest.prevention = prevention
    pest.isFavorite = false
    try? viewContext.save()
    presentationMode.wrappedValue.dismiss()
}
}
...

```

📖 Lesson 2.8: Journal Tab – Binary Data + Detail View (45 minutes)

2.8.1 JournalTabView

```

```swift
// Views/JournalTabView.swift
import SwiftUI
import CoreData

struct JournalTabView: View {
 @EnvironmentObject private var viewModel: FarmViewModel

```

```

@Environment(\.managedObjectContext) private var viewContext

@FetchRequest(
 entity: JournalEntry.entity(),
 sortDescriptors: [NSSortDescriptor(keyPath: \JournalEntry.date,
 ascending: false)]
) var entries: FetchedResults<JournalEntry>

@State private var showingAddEntry = false

var body: some View {
 NavigationView {
 List {
 ForEach(entries, id: \.self) { entry in
 NavigationLink(destination: JournalDetailView(entry:
 entry, viewModel: viewModel)) {
 JournalRowView(entry: entry, viewModel: viewModel)
 }
 }
 .onDelete(perform: deleteEntries)
 }
 .navigationTitle("កំណត់ហេតុកសិកម្ម")
 .toolbar {
 ToolbarItem(placement: .navigationBarTrailing) {
 Button(action: { showingAddEntry = true }) {
 Image(systemName: "plus")
 }
 }
 }
 .sheet(isPresented: $showingAddEntry) {
 AddJournalEntryView()
 .environment(\.managedObjectContext, viewContext)
 .environmentObject(viewModel)
 }
 }
}

private func deleteEntries(offsets: IndexSet) {
 for index in offsets {
 viewContext.delete(entries[index])
 }
 try? viewContext.save()
}
}
...

```

### **\*\*2.8.2 JournalRowView + JournalDetailView\*\***

```

```swift
struct JournalRowView: View {
    let entry: JournalEntry
    let viewModel: FarmViewModel

    var body: some View {

```

```

VStack(alignment: .leading, spacing: 6) {
    HStack {
        if let date = entry.date {
            Text(viewModel.formatDate(date)).font(.headline)
        }
        Spacer()
        if let weather = entry.weather, !weather.isEmpty {
            Text(weather).font(.caption).foregroundColor(.blue)
        }
    }
    if let content = entry.content, !content.isEmpty {
        Text(content)
            .font(.subheadline)
            .foregroundColor(.gray)
            .lineLimit(2)
    }
    if let location = entry.location, !location.isEmpty {
        Label(location, systemImage: "mappin.circle")
            .font(.caption)
            .foregroundColor(.gray)
    }
}
.padding(.vertical, 4)
}
}

```

```

struct JournalDetailView: View {
    let entry: JournalEntry
    let viewModel: FarmViewModel

    var body: some View {
        List {
            if let date = entry.date {
                Section(header: Text("កាលបរិច្ឆេទ")) {
                    Text(viewModel.formatDate(date))
                }
            }
            if let weather = entry.weather {
                Section(header: Text("អាកាសធាតុ")) { Text(weather) }
            }
            if let location = entry.location {
                Section(header: Text("ទីតាំង")) { Text(location) }
            }
            if let content = entry.content {
                Section(header: Text("មាតិកា")) { Text(content) }
            }
        }
        .navigationTitle("កំណត់ហេតុ")
    }
}

```

****2.8.3 AddJournalEntryView****

```

```swift
struct AddJournalEntryView: View {
 @Environment(\.presentationMode) var presentationMode
 @Environment(\.managedObjectContext) private var viewContext

 @State private var content = ""
 @State private var weather = ""
 @State private var location = ""
 @State private var date = Date()

 let weatherOptions = ["☀️ ថ្ងៃរះ", "☁️ មេឃពាក់កណ្តាល", "🌧️ ភ្លៀង", "🌤️ មេឃច្រើនពពក", "🌩️ ព្យុះ"]

 var body: some View {
 NavigationView {
 Form {
 Section(header: Text("កាលបរិច្ឆេទ")) {
 DatePicker("", selection: $date, displayedComponents:
 .date)
 .labelsHidden()
 }
 Section(header: Text("អាកាសធាតុ")) {
 Picker("អាកាសធាតុ", selection: $weather) {
 Text("ជ្រើសរើស").tag("")
 ForEach(weatherOptions, id: \.self) {
 Text($0).tag($0)
 }
 }
 }
 Section(header: Text("ទីតាំង")) {
 TextField("ឧ. វាលស្រែ ១", text: $location)
 }
 Section(header: Text("មាតិកា")) {
 TextEditor(text: $content)
 .frame(minHeight: 120)
 }
 }
 .navigationTitle("បន្ថែមកំណត់ហេតុ")
 .navigationBarItems(
 leading: Button("បោះបង់") {
 presentationMode.wrappedValue.dismiss()
 },
 trailing: Button("រក្សាទុក") { saveEntry()
 }.disabled(content.isEmpty)
)
 }
 }
}

```



```

private func saveEntry() {
 let entry = JournalEntry(context: viewContext)
 entry.id = UUID()
 entry.date = date
 entry.content = content
 entry.weather = weather
 entry.location = location
 try? viewContext.save()
 presentationMode.wrappedValue.dismiss()
}
}
...

```

---

## ## 🏠 Week 2 Mini-Project Assignment

### ### Task: Implement Core Data Persistence

#### **\*\*Requirements:\*\***

##### **\*\*1. Core Data Setup (20%):\*\***

- [ ] Create Data Model file with all 4 entities
- [ ] Generate or create NSManagedObject subclasses
- [ ] Set Codegen to "Manual/None" for manually written files
- [ ] Set up Core Data stack in App file

##### **\*\*2. CRUD Operations (40%):\*\***

- [ ] Implement Create (add new transactions)
- [ ] Implement Read (display with @FetchRequest)
- [ ] Implement Update (edit existing transactions)
- [ ] Implement Delete (swipe to delete)

##### **\*\*3. Finance Tab Completion (30%):\*\***

- [ ] Add Transaction form works
- [ ] Summary cards show correct totals
- [ ] Filter by income/expense works
- [ ] Data persists after app restart

##### **\*\*4. Sample Data & Debug (10%):\*\***

- [ ] Add option to load sample data
- [ ] Verify data is saved to device

#### **\*\*Bonus Challenge (Optional):\*\***

- Add search functionality to filter transactions
- Implement charts showing income vs expense
- Add export to CSV feature

---

### ### Submission Checklist:

- [ ] Core Data model created with all 4 entities
- [ ] Can add, view, edit, and delete transactions

- [ ] Data persists after closing and reopening app
- [ ] Summary cards update automatically
- [ ] Filter functionality works correctly
- [ ] No crashes when saving/loading data

---

### ### Common Core Data Issues & Solutions

Issue	Solution
"Cannot load model"	Check model file name matches `NSPersistentContainer(name:)` exactly
"Multiple commands produce"	Set entity Codegen to "Manual/None" in data model editor
Data not saving	Call `try context.save()` after changes
@FetchRequest not updating	Ensure View has access to correct context
Crash when deleting	Check for duplicate deletions
Predicate not working	Verify attribute names match exactly

---

### ### Key Differences: SwiftData vs Core Data

Feature	SwiftData (iOS 17+)	Core Data (iOS 13+)
Syntax	`@Model` macro	`NSManagedObject` subclasses
Query	`@Query`	`@FetchRequest`
Setup	`modelContainer()`	`NSPersistentContainer`
ViewModel	`@Observable`	`ObservableObject`
Pass to views	`environment(vm)`	`environmentObject(vm)`
Read in views	`@Environment(VM.self)`	`@EnvironmentObject var vm: VM`
Navigation	`NavigationStack`	`NavigationView`

---

*\*End of Week 2 Materials – Core Data for iOS 13+\**